HTMX実践チュートリアル — ServletとつくるHDA Webアプリ

> **対象**: HTMX未経験者／HTML・CSSの基礎を理解し、Java Servletを今後利用する開発者
> **ゴール**: 小さなタスク管理アプリを段階的に作り、HTML断片による更新、検索、CRUD、HTTP、HDA設計、Servlet/JSPへの接続を説明・実装できる状態になる
> **使用版**: htmx **2.0.11**／**JDK 25**／**Apache Tomcat 11**（Jakarta Servlet **6.1**・Jakarta Pages/JSP **4.0**）／**Maven**／**VS Code**
> **開発環境**: Windows 11、Extension Pack for Java、Community Server Connector、既存Maven WARプロジェクト `multi-management-system`
> **構築方針**: 既存のGitリポジトリで `main` からローカル学習ブランチ `study/htmx-hda` を作成し、そのブランチだけで教材を実装する。新規Mavenプロジェクトは作らず、既存の `index.jsp`・画面・Servletは変更しない。学習終了後は `main` **にマージせず学習ブランチを削除**する
> **執筆基準**: 2026年10月。htmx 4.0は既に公開されているが、本教材は安定した2.x系列に固定し、両版の構文・動作を混在させない

この教材の使い方

最初から全コードを理解する必要はない。**最初にLesson 02で学習専用ブランチを作り、`main` のままではコードを書かない**。サーバーは付録のServletをそのまま起動し、各章でHTML側のコードを少しずつ追加・変更する。その後、サーバーから返るHTMLとServletの対応関係を確認する。**終了時は付録Eに沿って学習ブランチを削除**する。

各Lessonは次の順序で進める。

1. **作るもの**: ボタン、入力フォームなど、目に見える機能
2. **記述**: 実際のHTMLとHTMX属性
3. **通信**: 発生するHTTPリクエストとServletのレスポンス
4. **理解**: 新しい構文の意味と注意点
5. **確認**: DevToolsやブラウザーで動作を確かめる

コード内の相対URLは、原則として **`http://localhost:8080/multi-management-system/htmx-lab/index.html`** からの相対URL。例えば `api/hello` は **`http://localhost:8080/multi-management-system/htmx-lab/api/hello`** を表す。既存アプリのコンテキストパスは `/multi-management-system`、教材の専用パスは `/htmx-lab` と区別する。

`index.jsp` はチームの既存入口としてそのまま残す。教材画面は `src/main/webapp/htmx-lab/index.html` に隔離し、タスク管理用Servletのマッピングも `/htmx-lab/api/*` に限定する。各Lessonの操作を既存画面へ無断で追加しない。

> **重要**: 各LessonのHTMLは学習対象部分を抜き出したもの。最終的な動作を保証する完成版HTMLとServletは**付録A〜C**に全文掲載する。途中で構成が分からなくなったら完成版と比較する。

学習マップ

段階	到達点	主な技術
0. 土台	専用ブランチを作りサーバーを起動	Gitブランチ、HTTP、Tomcat、Servlet
1. 読み込み	ボタンから表示内容を取得	`hx-get`、`hx-target`、`hx-swap`
2. 変更	追加・削除・更新	`hx-post`、フォーム、HTML断片
3. 操作性	検索・絞り込み・編集	`hx-trigger`、`hx-include`、`hx-sync`
4. 実用化	待機表示・検証・複数箇所更新	`hx-indicator`、イベント、HTTPステータス
5. 設計	HDA・Servlet/JSP連携	SSR、履歴、認証、CSRF、XSS、MVC

Part 0 — HTMXを使う前に

Lesson 01 — HDAとは何か

1.1 MPA・SPA・HDA

MPA (Multi-Page Application)

- リンク・フォーム操作によってサーバーにリクエストを送る
- サーバーがHTMLを作る
- ブラウザーはページ全体を再表示するのが基本

SPA (Single-Page Application)

- JavaScriptが画面状態を大きく管理する
- サーバーからJSONを取得し、JavaScriptがDOMを組み立てる方式が多い
- React等はSPAの構築手段になり得るが、SPAだけに限定されるわけではない

HDA (Hypermedia-Driven Application)

- サーバーが返すHTMLを主要な画面表現として扱う
- HTML上の宣言的な属性でHTTP通信と画面更新を記述する
- ページ全体の再読込を減らしつつ、サーバー主導のシンプルな構成を維持する
- JavaScriptは必要な箇所に限定して追加できる

HDAは「HTMXを使ったSPA」の別名ではない。**画面表現の中心がサーバー生成HTMLにあり、そのHTMLが次の操作を指示する**、という設計が重要になる。

```
従来のフォーム送信
ブラウザー ── POST /tasks ──> Servlet
ブラウザー <── ページ全体HTML ── Servlet
ブラウザー → ページ全体を再描画

HTMXによる送信
ブラウザー ── POST /api/add ──> Servlet
```

```
ブラウザー ←— <li>...</li>等のHTML —— Servlet
HTMX → 指定部分だけを置き換え
```

1.2 HTMXが担当する仕事

HTMXは**HTML属性を読み取り、HTTPリクエストを送り、返されたHTMLを指定の場所に挿入する**ライブラリ。

```
<button hx-get="api/hello" hx-target="#result">
  挨拶を取得
</button>
<div id="result"></div>
```

押すと次の通信が発生する。

```
GET /multi-management-system/htmx-lab/api/hello HTTP/1.1
HX-Request: true
```

Servletが次の断片を返す。

```
<p>ServletからHTMLが届いた！</p>
```

結果のDOMは次の形になる。

```
<div id="result">
  <p>ServletからHTMLが届いた！</p>
</div>
```

`fetch()`、`JSON.parse()`、`innerHTML = ...` を自分で記述せずに実現できる。

誤解しやすい点

- HTMXはServletやJSPを置き換えない
- HTMXはデータベースを直接操作しない
- HTMXはブラウザー内で動くJavaScriptライブラリ。サーバー側JavaScriptは不要
- すべての機能をHTMXだけで実現する必要はない
- HTMXの入力値やリクエストヘッダーを信用してはいけない

1.3 まず覚える4つ

属性	役割	読み方
`hx-get="URL"`	GETを発行	どこから取得するか

属性	役割	読み方
`hx-post="URL"`	POSTを発行	どこに送るか
`hx-target="セレクター"`	HTMLを適用する要素	どこを変えるか
`hx-swap="方式"`	HTMLの適用方法	どのように変えるか

後で `hx-trigger`（いつ通信するか）を加えると、基本構文が完成する。

理解チェック

- サーバーが返すのは、必ずJSONでなければならないか → **いいえ**。通常はHTML
- `hx-get` はどこで実行されるか → **ブラウザー**
- タスクの正しい状態を管理するのは誰か → **サーバー／DB**

Lesson 02 — 構築済みのVS Code・Tomcat 11へ教材を追加する

2.1 構築済み環境を確認

添付の「環境構築.md」で導入済みの環境をそのまま使用する。**JDKやTomcatの再インストール、新しいMavenプロジェクトの作成は不要**。

項目	現在の環境	教材での用途
OS	Windows 11	VS Code・ブラウザー・Git Bash
JDK	25	JavaのコンパイルとTomcatの実行
Webコンテナ	Apache Tomcat 11	Jakarta Servlet 6.1・JSP 4.0
IDE	VS Code / Extension Pack for Java	Java、JSP、HTML編集
サーバー連携	Community Server Connector	Exploded デプロイ・Publish
ビルド	インストール済みMaven	WARパッケージ作成
Mavenプロジェクト	`team.multi.project:multi-management-system`	既存アプリへ機能追加

VS Codeで既存の `multi-management-system` を開き、Git Bashターミナルで確認する。この段階ではまだ学習用ファイルを編集しない。

```
java -version
javac -version
mvn -version
```

`java` と `javac` は25、`mvn -version` 内のJavaの参照先もJDK 25になっていることを確認する。異なる場合は `JAVA_HOME` と `Path` を修正してVS Codeを再起動する。Tomcat側のJDK設定もJDK 25を指すようにする。

バージョン対応: Tomcat 11はJakarta Servlet 6.1、JSP 4.0を実装し、Java 17以降で動く。本教材ではすべてJDK 25に統一する。`jakarta.servlet.*` を使い、旧 `javax.servlet.*` と混在させない。

2.2 学習専用のGitブランチを作成する

目的: `main` のチーム開発に学習用の変更を混ぜず、学習が終わったら変更をまとめて破棄できるようにする。

```
main ──●──────────────────────────── チーム開発用（変更しない）
        \
         ●──●──●   study/htmx-hda    HTMX学習専用
         学習コミット       └── 終了後はマージせず削除
```

開始前の確認 — VS CodeのGit Bashを **`pom.xml` のある `multi-management-system` フォルダ**で開く。本教材の `git add` に書いた相対パスも、このフォルダが基準。Gitリポジトリのルートが1階層上にあっても、この場所からGitコマンドを実行できる。

```
git status --short
git branch --show-current
```

- `git status --short` が**何も表示しない**ことを確認。すでに別作業の未コミット変更がある場合は、まずその変更の保存先を決める。**未コミットのままブランチを切り替えると、その変更が移動先ブランチにも残る場合がある**
- 既存の学習ブランチがある場合は新規作成せず、`git switch study/htmx-hda` で再開する

新規作成 — 作業ツリーがクリーンであることを確認してから実行する。

```
git switch main
git pull --ff-only origin main
git switch -c study/htmx-hda
git branch --show-current
```

最後に `study/htmx-hda` と表示されたら準備完了。`git pull --ff-only origin main` はチームの最新変更を取り込む操作であり、学習コードを `main` にコミットする操作ではない。`origin` が未設定またはネットワーク接続がない場合は `git pull` を省略し、手元の `main` を起点にする。`study/htmx-hda` が既に存在する場合は `git switch -c` を繰り返さない。

以後のルール

1. ファイル作成・`pom.xml` 編集・Servlet実装・ビルドはすべて `study/htmx-hda` 上で行う
2. 作業開始時・コミット前に `git branch --show-current` で現在のブランチを確認する
3. 学習中の変更を `main` にマージしない。Pull Requestも作成しない。GitHubへプッシュする必要もない
4. 学習の区切りでは**ローカルコミット**してよい。`main` の履歴には追加されない
5. 学習終了時は**付録E**の手順でブランチ・ビルド成果物・デプロイ状態を整理する

学習ブランチ内でのコミット例:

```
git status --short
git add pom.xml src/main/java/team/multi/project/tutorial/ src/main/webapp/htmx-lab/
git diff --cached --stat
git commit -m "study: practice htmx with servlet"
```

`git add` の対象は教材用ファイルと、実際に教材のために編集した設定だけに限定する。既存の `web.xml` も必要に応じて追跡する。**別の人の変更を一括で `git add .` しない**。このコミットは学習の復元地点として使えるが、ブランチを削除すると通常のブランチ参照からたどれなくなる。

ここから先は必ず学習ブランチ上で作業する。

2.3 既存プロジェクト内に2ファイルを追加

`git branch --show-current` が `study/htmx-hda` であることを確認してから、**既存の** `multi-management-system` に次を追加する。新しいプロジェクトは作らない。

```
multi-management-system/              ← 既存のMavenプロジェクト
├── pom.xml                            ← 必要な設定のみ追記（置換しない）
├── src/
│   └── main/
│       ├── java/
│       │   └── team/
│       │       └── multi/
│       │           └── project/
│       │               └── tutorial/
│       │                   └── TaskServlet.java  ← 追加
│       └── webapp/
│           ├── index.jsp              ← 既存のまま
│           ├── htmx-lab/
│           │   └── index.html         ← 追加
│           └── WEB-INF/               ← 既存のまま
└── target/                            ← Mavenがビルド時に生成
    └── multi-management-system/       ← Explodedデプロイ対象
```

`team.multi.project.tutorial` はJavaのパッケージ名。MavenのGroupId `team.multi.project` と整合させているが、パッケージ名は教材用の例であり、既存チームコードの変更は不要。

- `TaskServlet.java`: **付録B**を全文コピー
- `htmx-lab/index.html`: 最初はこのLessonの最小版を使い、最終的に**付録C**の完成版をコピー
- `pom.xml`: **付録Aの「既存プロジェクトへの追記」**に従って必要な要素のみ追加

2.4 `pom.xml` は上書きせずに設定を統合する

学習ブランチの `pom.xml` を編集する。Mavenプロジェクトにはすでに `pom.xml` があるため、`groupId`・`artifactId`・既存の依存ライブラリ・既存プラグイン設定を保持し、次の状態になるよう**不足部分のみ**追

加・修正する。学習ブランチでの編集は `main` の同ファイルへ反映されない。

設定	値	理由
`groupId`	`team.multi.project`	既存のGroupId
`artifactId`	`multi-management-system`	既存のArtifactId
`packaging`	`war`	TomcatへWebアプリとしてデプロイ
`maven.compiler.release`	`25`	JDK 25でコンパイル
`jakarta.servlet-api`	`6.1.0` / `provided`	Tomcat 11がServletを実行時に提供
`build.finalName`	`multi-management-system`	環境構築手順のExplodedフォルダ名に統一

> `pom.xml` に `<properties>`、`<dependencies>`、`<build>` が既にある場合、**同じ要素を二重に作らない**。既存の要素内へ必要な行を統合する。既存にコンパイラプラグインがあればバージョン・設定の整合性を確認する。

2.5 教材用Servletの配置とURL

`TaskServlet.java` に**付録B**の全文を配置する。Servletの先頭は次の形。

```
package team.multi.project.tutorial;

// importは付録B参照
@WebServlet("/htmx-lab/api/*")
public final class TaskServlet extends HttpServlet {
    // ...
}
```

Webアプリ全体のコンテキストパスにServletのマッピングを結合すると、リクエスト先は次のようになる。

```
コンテキストパス: /multi-management-system
Servletマッピング: /htmx-lab/api/*
最終的なURL:    /multi-management-system/htmx-lab/api/hello
```

ブラウザーに送るAPIリクエストは次のとおり。Servletが**HTML断片**を返す。

HTTP	相対パス	機能
GET	`api/hello`	挨拶HTML
GET	`api/clock`	サーバー時刻
GET	`api/tasks`	タスクの `<li>` 一覧
GET	`api/count`	件数
GET	`api/edit?id=1`	編集フォーム

HTTP	相対パス	機能
GET	`api/view?id=1`	通常のタスク行
GET	`api/preview?title=...`	入力プレビュー
GET	`api/validate?title=...`	入力検証
GET	`api/empty`	HTTP 204
GET	`api/failure`	HTTP 500
POST	`api/add`	タスク追加
POST	`api/toggle?id=1`	完了状態切り替え
POST	`api/delete?id=1`	タスク削除
POST	`api/save?id=1`	タイトル更新

データはServletの**メモリ内**で管理するため、Tomcatや教材Webアプリの再起動・再デプロイで初期化される。既存のMySQLや生徒情報を操作しない。

2.6 最小の `src/main/webapp/htmx-lab/index.html`

```
<!doctype html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>HTMX練習</title>
  <script src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.11/dist/htmx.min.js"
          integrity="sha384-
2OatzQy1H+Zd/IIrjr1TcuDGqLXeHhbooAyJY1KdQMKnr4LZ22k31GBLdYKHmVjg"
          crossorigin="anonymous" defer></script>
</head>
<body>
  <main>
    <h1>HTMX練習</h1>
    <button hx-get="api/hello" hx-target="#result">挨拶を取得</button>
    <div id="result"></div>
    <!-- 各LessonのHTMLをここに追加 -->
  </main>
</body>
</html>
```

`api/hello` は `index.html` が置かれた `/htmx-lab/` ディレクトリを基準に解決される。`/api/hello` のように先頭へ `/` を付けるとコンテキストパスが外れ、別URLになるため注意。

2.7 ビルド・デプロイ（添付の環境構築手順と同じ）

1. `git branch --show-current` が `study/htmx-hda` であることを確認し、VS Codeのターミナルで既存の `pom.xml` がある `multi-management-system` フォルダへ移動

2. 初回はTomcatを停止して `mvn clean package` を実行、以降のビルドは `mvn package`
3. `BUILD SUCCESS` を確認し、`target/multi-management-system/` が存在することを確認
4. VS Codeの「表示 → ビューを開く → Servers」で `Tomcat 11` を表示
5. Tomcat 11を右クリック → **Add Deployment** → **Exploded**
6. `multi-management-system/target/multi-management-system` を選択（**`htmx-lab` のサブフォルダだけを選ばない**）
7. 必要に応じて **Publish Server (Full)** を実行してからTomcatを起動
8. ブラウザーから次のURLへアクセス

```
教材画面: http://localhost:8080/multi-management-system/htmx-lab/index.html
確認API: http://localhost:8080/multi-management-system/htmx-lab/api/hello
既存画面: http://localhost:8080/multi-management-system/index.jsp
```

教材画面で「挨拶を取得」を押し、「ServletからHTMLが届いた！」と表示されれば成功。`api/hello` を直接開くと `<p>` のような断片しか返らないことも確かめる。

更新時: Java・HTML・JSPを変更したら再度 `mvn package` し、Serversパネルから必要に応じて **Publish Server** または **Publish Server (Full)** を実行し、ブラウザーを再読込する。**Tomcatの停止・起動だけで新しい `target/` の内容が確実に反映されるとは限らない**。

注意: `index.html` を `file://` で直接開かない。HTMXのリクエストはTomcatのURLで確かめる。外部CDNへ接続できない環境では、htmx JavaScriptを `src/main/webapp/htmx-lab/js/` に保存し、`src="js/htmx.min.js"` に変更する。

2.8 エラーが出たとき

症状	先に確認すること
`404`（`index.html`）	`src/main/webapp/htmx-lab/index.html` とExplodedの選択先
`404`（`api/hello`）	`@WebServlet("/htmx-lab/api/*")`、コンパイル結果、Publish状態、`web.xml` の設定
`ClassNotFoundException: javax.servlet...`	Tomcat 11では `jakarta.servlet.*` を使用。旧ライブラリの混在を確認
Javaのコンパイルエラー	Mavenの実行JDKが25か、`maven.compiler.release` が25か
`mvn` コマンドが見つからない	Maven `bin` へのPath、VS Codeの再起動
ページは出るがHTMXが反応しない	ブラウザーDevToolsのConsoleとNetworkでCDN読み込みを確認
`target/multi-management-system` がない	`<build><finalName>` の設定、`mvn package` の成功を確認
既存アプリとURLが衝突する	教材ServletはWebアプリ内の `/htmx-lab/api/*` のみに登録する

古いMavenアーキタイプによる `web.xml` が残っていても、**既存アプリで使用している場合は削除・全置換しない**。アノテーションからServletが登録されない場合は、`web.xml` の `metadata-complete` やServletマッピングを調べる。必要なら既存の `web.xml` の `<web-app>` 内に**以下を追記**し、アノテーション側と同じURLを二重登録しないよう整理する。

```
<servlet>
  <servlet-name>HtmxTutorialTaskServlet</servlet-name>
  <servlet-class>team.multi.project.tutorial.TaskServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>HtmxTutorialTaskServlet</servlet-name>
  <url-pattern>/htmx-lab/api/*</url-pattern>
</servlet-mapping>
```

この方法はアノテーションが適切に検出されない場合の代替策。最初は**付録Bの `@WebServlet` のみ**で試す。

確認問題

1. `/multi-management-system` と `/htmx-lab` の役割はどう違うか
2. ブラウザーの `api/hello` はどのURLへ解決されるか
3. `mvn package` と `Publish Server` がそれぞれ行う処理は何か
4. `jakarta.servlet-api` の `provided` が必要な理由は何か

Part 1 — HTMLを部分更新する

Lesson 03 — ボタンからHTMLを取得する: `hx-get`

作るもの: 「挨拶を取得」を押すとサーバーからメッセージを受信する機能。

`<main>` の中へ追加する。

```
<button type="button" hx-get="api/hello">
  挨拶を取得
</button>
```

動作

1. ボタンクリック
2. GET `/api/hello`
3. Servletが `<p>ServletからHTMLが届いた！</p>` を返す
4. HTMXがレスポンスをボタンの**内側**へ挿入する

結果としてボタン自体の中身が変わる。これは既定の更新対象が**リクエスト元の要素自身**、既定の更新方法が `innerHTML` だから。

調査: 開発者ツール → Network → `hello` → Headers と Response を確認。`HX-Request: true` が見えるか確認する。

演習: ボタンを2つ作り、それぞれ `api/hello` と `api/clock` を読み込ませる。

Lesson 04 — 更新先を指定する: `hx-target`

作るもの: ボタンを維持し、メッセージだけを下へ表示。

```
<button type="button"
        hx-get="api/hello"
        hx-target="#message">
  挨拶を取得
</button>

<div id="message">ここに表示</div>
```

構文: `hx-target="#message"` はCSSセレクター。`id="message"` の要素を対象とする。

実行後:

```
<div id="message">
  <p>ServletからHTMLが届いた！</p>
</div>
```

ターゲット指定方法の例:

値	対象
`#message`	IDが `message` の要素
`.result`	クラス `result` の要素（通常、最初の一致要素）
`this`	リクエストを発行する要素自身
`closest li`	最も近い祖先の `li`（自身も候補）
`closest tr`	最も近い表の行
`next .result`	後続にある一致要素
`previous .result`	前方にある一致要素
`find .label`	リクエスト元の子孫にある一致要素

演習: それぞれのボタンが別々の `div` だけを変更する画面を作る。

Lesson 05 — 置き換え方を選ぶ: `hx-swap`

作るもの: 同じレスポンスをさまざまな方法で挿入する。

```
<button hx-get="api/hello" hx-target="#area" hx-swap="innerHTML">
  内側を更新
</button>
<button hx-get="api/hello" hx-target="#area" hx-swap="outerHTML">
  要素ごと置換
</button>
<div id="area">更新対象</div>
```

hx-swap	効果	主な用途
innerHTML	対象の中身を入れ替え	検索結果、一覧（既定）
outerHTML	対象自身を置き換え	行、カード、編集フォーム
beforebegin	対象の直前に追加	前置き通知
afterbegin	対象の最初の子として追加	最新コメントを先頭へ
beforeend	対象の最後の子として追加	追加ロード、チャット
afterend	対象の直後に追加	続きを表示
delete	対象を削除	返却HTML不要の削除UI
none	通常の対象は変更しない	イベントやOOB更新のみ
textContent	文字列として挿入	HTMLを解釈させたくない表示

落とし穴: `outerHTML` で `#area` を `<p>` に置換すると、その `id` はなくなる。次に `#area` へ送ると対象が見つからない。継続利用する領域ではIDを維持するHTMLを返すか `innerHTML` を選ぶ。

```
<!-- 目的: liを丸ごと交換 -->
<li id="task-1">
  <button hx-get="api/edit?id=1"
          hx-target="closest li"
          hx-swap="outerHTML">編集</button>
</li>
```

演習: `afterbegin` と `beforeend` を切り替え、表示順序の差を確認する。

Lesson 06 — 通信のきっかけを変える: `hx-trigger`

`hx-trigger` はリクエストの**発火条件**を指定する。省略すると要素に応じた既定イベント（ボタンなら `click`、フォームなら `submit` など）が使われる。

作るものA: ページ表示時に時刻を取得

```
<div hx-get="api/clock"
     hx-trigger="load">
  時刻を読み込み中…
</div>
```

作るものB: クリックしなくても検索

```
<label for="query">プレビューする文字列</label>
<input id="query" name="title"
       hx-get="api/preview"
       hx-trigger="input changed delay:400ms"
       hx-target="#preview">
<div id="preview"></div>
```

リクエスト例:

```
GET /multi-management-system/htmx-lab/api/preview?title=ABC
HX-Request: true
```

`input` は入力イベント、`changed` は値が変わったときだけ、`delay:400ms` は最後の入力から400ms待つことを表す。

作るものC: 一定間隔で更新

```
<div hx-get="api/clock"
     hx-trigger="load, every 10s">
  時刻読み込み中…
</div>
```

指定	意味
`click`	クリック時
`change`	値が確定・変更されたとき
`input`	入力中
`keyup`	キーを離したとき
`submit`	フォーム送信時
`load`	要素が処理されたとき
`revealed`	スクロールで要素が表示されたとき
`intersect once`	可視領域に入った最初の1回

指定	意味
`every 10s`	10秒ごと
`click once`	最初のクリック1回だけ
`input changed delay:400ms`	変更が止まって400ms後
`customEvent from:body`	`body` に伝播する独自イベントを受けたとき

注意: `every 10s` はブラウザーが継続してリクエストする。更新が必要な要素だけに使用し、サーバー負荷を検討する。

演習: 時計を10秒ごとではなく5秒ごとに変更し、Networkで通信回数を確認する。

Lesson 07 — HTTPを理解する: フォーム送信とパラメーター

HTMXの属性を覚える前に、HTMLフォームの基本を確かめる。

```
<form action="api/add" method="post">
  <label for="task-name">タイトル</label>
  <input id="task-name" name="title" required>
  <button type="submit">追加</button>
</form>
```

フォームは `name="title"` に入力された値をリクエストに含める。`id` は画面上の識別、`name` は送信するパラメーター名。`name` がなければ通常、その値は送られない。

```
title=Java%E3%81%AE%E5%AD%A6%E7%BF%92
```

Servlet側:

```
String title = request.getParameter("title");
```

GETとPOSTの使い分け

	GET	POST
主な目的	参照、検索、フィルター	作成、変更、削除等
値の位置	主にURLクエリ	通常リクエストボディ
ブックマーク	検索条件をURLで表現しやすい	通常しない
再実行	原則、状態を変更しない	状態を変更し得る
安全性	**GET自体は秘密通信ではない**	**POSTもHTTPSがなければ安全でない**

HTMXでも**状態を変更する処理をGETに置かない**。削除は `hx-get` ではなく `hx-post` や `hx-delete` を使う。

HTTPの約束

- `200 OK`: 処理成功
- `204 No Content`: 成功したが表示する本文なし
- `400 Bad Request`: 不正な入力
- `401 Unauthorized`: 未認証
- `403 Forbidden`: 権限なし
- `404 Not Found`: 対象なし
- `422 Unprocessable Content`: 形式は正しいが内容が妥当でない
- `500 Internal Server Error`: サーバー側エラー

次章からフォームを部分更新へ発展させる。

Part 2 — タスク管理アプリを作る

Lesson 08 — 一覧をサーバーから読み込む

作るもの: 初期表示でタスク一覧を取得する。

```
<h2>タスク一覧</h2>
<ul id="task-list"
    hx-get="api/tasks"
    hx-trigger="load">
  <li>読み込み中…</li>
</ul>
```

Servletから返るHTMLの例:

```
<li id="task-2" class="task-row">
  <span class="task-title">hx-getを使ってみる</span>
  <!-- 完了・編集・削除ボタン -->
</li>
<li id="task-1" class="task-row">
  <span class="task-title">HTMLの仕組みを復習する</span>
</li>
```

このレスポンスは**`<html>` も `<body>` も持たない断片**。`<ul>` の中にそのまま挿入される。

設計の分担

- Servlet: タスク一覧を読み、HTML断片を生成
- HTMX: GETを送信し、届いた `<li>` を挿入
- CSS: `<li>` の見た目を調整

実際のServletの該当箇所:

```
case "/tasks" -> html(res, rows(req));
```

`rows(req)` は`<li>`を連結して返すメソッド。ServletがJSONではなくHTMLを返す点が中心。

演習: 初期文字列を `現在のタスクを取得中` に変え、読み込み完了後に置換されることを確認する。

Lesson 09 — タスクを追加する: `hx-post`

作るもの: フォーム送信後、ページ遷移せず一覧が変わる。

```
<form hx-post="api/add"
      hx-target="#task-list">
  <label for="title">タスク名</label>
  <input id="title" name="title" required maxlength="80">
  <button type="submit">追加</button>
</form>
```

このフォームをLesson 08の `<ul>` より前へ置く。

リクエスト

```
POST /multi-management-system/htmx-lab/api/add HTTP/1.1
Content-Type: application/x-www-form-urlencoded
HX-Request: true

title=Servlet%E3%82%92%E5%AD%A6%E3%81%B6
```

サーバーは追加処理後、**新しいタスク一覧の `<li>` 群**を返す。既定の `hx-swap="innerHTML"` によりリスト全体の中身が置き換わる。

ここでの設計判断

- 新規要素1件を `afterbegin` で追加する方法もある
- 本教材では検索や絞り込みとの整合を優先し、変更後に一覧の中身を丸ごと再生成する
- 小規模アプリでは分かりやすさを優先。大量データの場合は必要箇所のみを更新する構成も検討する

入力欄を送信成功後に空にするには、フォームへ追加する。

```
<form hx-post="api/add"
      hx-target="#task-list"
      hx-on::after-request="if (event.detail.successful) this.reset()">
```

`hx-on::after-request` はHTMXイベントを扱う書き方。成功時だけ `form.reset()` を実行する。**この1行はJavaScript**。HTMXはJavaScriptを排除するのではなく、必要な補助処理を少量書けるようにする。

確認: 追加後、ブラウザーを再読み込みしてもタスクが残るかを確認。値の保持先はDOMではなくServletのメモリ。

演習: `name="title"` を `name="task"` に変更して、追加できなくなる理由を説明する。元へ戻すこと。

Lesson 10 — タスクを削除する: 送信先と対象

作るもの: 行の「削除」で、そのタスクだけをサーバーから削除。

Servletが生成した各 `<li>` に次のボタンが含まれている。

```
<button type="button"
        hx-post="api/delete?id=1"
        hx-target="#task-list"
        hx-confirm="このタスクを削除する？">
  削除
</button>
```

`hx-confirm` は送信前に標準の確認ダイアログを出す。

処理の流れ:

1. 確認で「OK」を選択
2. `POST /api/delete?id=1`
3. ServletがID 1のタスクを削除
4. 削除後の一覧HTMLを返す
5. `#task-list` の中身を更新

削除方法の別案

```
<button hx-post="api/delete?id=1"
        hx-target="closest li"
        hx-swap="delete">
  削除
</button>
```

この方法はDOMの1行だけ削除する。ただし、サーバー側の**削除が成功した場合だけ**行を消す設計にすること。検索件数や絞り込み表示との同期も別途考える必要がある。本教材の完成版は**一覧再描画方式**を採用する。

補足: `hx-delete` は使えるか

```
<button hx-delete="api/tasks/1">削除</button>
```

htmxにはHTTP DELETEを送る属性もある。しかし、**Servlet側も `doDelete()` とURLルーティングを実装する必要がある**。完成版Servletは `POST api/delete` を実装しているため、この例をそのまま貼っても動かない。HTMLの属性名を変えただけではサーバーのAPIは増えない。

演習: キャンセルすると通信されないことをNetworkで確認する。

Lesson 11 — 完了／未完了を切り替える

作るもの: 各タスクの完了状態を反転。

```
<button type="button"
        hx-post="api/toggle?id=1"
        hx-target="#task-list">
  完了にする
</button>
```

Servlet:

```
// 簡略化した概念コード
// 既存タスクを読み取り、doneを反転して保存
// 更新された一覧HTMLを返す
```

実装の本体は付録Bの `/toggle` 部分。**完了状態の判定はブラウザー上の文字やCSSクラスではなく、サーバーが保持するデータを使う**。

返却されたリストは例えば次のようになる。

```
<li class="task-row done">
  <span class="task-title">HTMLの仕組みを復習する</span>
  <span class="state">完了</span>
  <button>戻す</button>
</li>
```

CSS例:

```
.done .task-title {
  text-decoration: line-through;
  opacity: .65;
}
```

これで表示もサーバー状態に一致する。

注意: 状態変更をHTMLの `onclick` だけで行うと、画面だけが変わりサーバー状態と矛盾することがある。

演習: 完了タスクの文字に取り消し線を付ける。CSSのみで実装する。

Lesson 12 — インライン編集: `closest` と `outerHTML`

作るもの: 「編集」を押した行だけ入力フォームに変わる。

通常表示:

```
<li id="task-1">
  <span>HTMLの仕組みを復習する</span>
  <button hx-get="api/edit?id=1"
          hx-target="closest li"
          hx-swap="outerHTML">編集</button>
</li>
```

レスポンス:

```
<li id="task-1">
  <form hx-post="api/save?id=1"
        hx-target="#task-list">
    <label>タイトル
      <input name="title"
             value="HTMLの仕組みを復習する" required>
    </label>
    <button type="submit">保存</button>
    <button type="button"
            hx-get="api/view?id=1"
            hx-target="closest li"
            hx-swap="outerHTML">取消</button>
  </form>
</li>
```

何が起きたか

- `hx-target="closest li"`: 編集ボタンを含む最も近い行を対象にする
- `hx-swap="outerHTML"`: その行を編集用HTMLへ丸ごと交換
- `<form hx-post="api/save...">`: 保存時にServletが更新処理
- 取消ボタン: サーバーから通常の行を再取得して戻す

重要: 挿入後のHTMLにある `hx-post` 等もHTMXが処理できる。通常のHTMXスワップで挿入された要素を自分で再登録する必要はない。

`outerHTML` を使う場合、返すHTMLのトップレベルを正しい `<li>` にする。`<ul>` の子として不正なDOM構造にしない。

演習: サーバー上のタイトルを変更して保存し、タスク一覧を再読み込みしても変更が残るか確認する。

Lesson 13 — ライブ検索: `hx-trigger` の応用

作るもの: 入力のたびにサーバーへ検索条件を送り、一覧だけ更新。

```
<label for="search">検索</label>
<input id="search" type="search" name="q"
       hx-get="api/tasks"
       hx-trigger="input changed delay:350ms, search"
       hx-target="#task-list">
```

Servletは `request.getParameter("q")` を読み、タスク名で絞り込む。

```
GET /multi-management-system/htmx-lab/api/tasks?q=Java
```

解説

- `type="search"`: 検索入力用の標準HTML
- `name="q"`: 検索パラメーター
- `input`: 入力時に発火
- `changed`: 値が変化したときだけ
- `delay:350ms`: 入力が落ち着くまで送信を遅延
- `search`: 検索欄のクリアなどによる検索イベントも扱う

デバウンス: 連続入力をまとめて無駄な通信を抑える技法。

追加演習

1. 検索結果がゼロの場合に `該当するタスクなし` と表示する（完成版Servletに実装済み）
2. `delay:350ms` を消してNetworkのリクエスト数を比較する
3. `hx-trigger="change"` に変更して体験の違いを見る

重要: デバウンスだけでは、既に送信した古いリクエストと新しいリクエストの競合は完全に解消しない。次章の `hx-sync` を組み合わせる。

Lesson 14 — 検索・状態フィルターを組み合わせる: `hx-include`

作るもの: キーワードと「すべて／未完了／完了」を同時に指定する。

```
<div>
  <label for="search">検索</label>
  <input id="search" name="q" type="search"
         hx-get="api/tasks"
         hx-trigger="input changed delay:350ms, search"
```

```
          hx-include="#status"
          hx-target="#task-list">

  <label for="status">状態</label>
  <select id="status" name="status"
          hx-get="api/tasks"
          hx-trigger="change"
          hx-include="#search"
          hx-target="#task-list">
    <option value="all">すべて</option>
    <option value="open">未完了</option>
    <option value="done">完了</option>
  </select>
</div>
```

検索欄を変更したときは `q` と `status`、選択欄を変更したときも `status` と `q` を送る。

```
/api/tasks?q=Java&status=open
```

要点

- `hx-include="#status"`: ほかのフォーム要素の値を追加する
- セレクターで選択される要素には原則 `name` が必要
- 初期表示の一覧にも2つの値を渡すと整合する

```
<ul id="task-list"
    hx-get="api/tasks"
    hx-trigger="load"
    hx-include="#search, #status"></ul>
```

完成版の追加・更新・削除ボタンも `hx-include="#search, #status"` を持つ理由: 変更後の一覧を現在の絞り込み条件に合わせて再描画するため。

その他のパラメーター制御

```
<button hx-post="api/any"
        hx-vals='{"mode":"quick"}'>実行</button>
```

`hx-vals` は追加データを指定する。上の例は説明用であり `api/any` は完成版Servletに存在しない。`hx-vals` の通常の値はJSON文字列。`js:` による動的評価はXSSリスクを増やすため、初学段階では不要。

```
<form hx-post="api/any" hx-params="not password">
```

`hx-params` は送信パラメーターを絞る。ただし、**ブラウザーで送信しない設定にしても機密データ保護・認可の代わりにはならない**。

Lesson 15 — 検索の競合を抑える: `hx-sync`

例えば、入力 `J` の検索が遅延し、そのあと `Java` の検索が先に応答することがある。最後に古い `J` の結果が返ると、画面が古い条件で上書きされるおそれがある。

```
<input id="search" name="q" type="search"
       hx-get="api/tasks"
       hx-trigger="input changed delay:350ms, search"
       hx-sync="this:replace"
       hx-target="#task-list">
```

`this:replace`: 同期対象をこの要素とし、すでに進行中のリクエストがあれば中断して新しいものへ置き換える。

検索入力と状態セレクトボックスの**両方**から届く一覧更新も競合し得る。完成版では両要素に `hx-sync="#task-list:replace"` を指定し、同じリスト要素を同期グループにして最後の要求を優先する。

戦略	挙動	用途
`drop`	処理中の新規リクエストを無視	二重送信抑制
`replace`	進行中のリクエストを中断して入れ替え	ライブ検索
`queue first`	最初の保留要求を保持	特定操作のキュー
`queue last`	最後の保留要求を保持	最終状態を優先
`queue all`	すべてを順に保持	各イベントが必須の場合
`abort`	ほかの要求を優先し、自分を中断	フォームの事前検証

補足: クライアントの通信中断は、サーバー処理の取り消しを保証しない。特に状態変更を伴う操作では、サーバー側の冪等性、競合制御、トランザクションが別途必要。

演習: Chrome DevToolsのNetworkで通信速度をSlow 3G等へ変え、入力中の動きを比較する。

Lesson 16 — 入力チェック: HTML5とServlet

作るもの: タイトル未入力や80文字超を受け付けない。

```
<form hx-post="api/add" hx-target="#task-list">
  <label for="title">タスク名</label>
  <input id="title" name="title"
         required minlength="1" maxlength="80">
```

```
  <button type="submit">追加</button>
</form>
```

標準のブラウザー検証で送信を止められる。ただし、ブラウザーのチェックは改変・回避できるためServletでも同じ条件を検証する。

```
String title = request.getParameter("title");
if (title == null || title.isBlank() || title.length() > 80) {
    response.setStatus(422);
    // エラーHTMLを返す
    return;
}
```

HTMX 2.x の重要な仕様: 既定ではHTTP `4xx`・`5xx` のレスポンスを通常のスワップ対象にしない。サーバーがエラーHTMLを返すだけでは画面に表示されないことがある。

簡単なエラー表示:

```
<p id="feedback" role="status" aria-live="polite"></p>
<script>
  document.body.addEventListener('htmx:responseError', function (event) {
    document.querySelector('#feedback').textContent =
      '入力または処理に失敗（HTTP ' + event.detail.xhr.status + '）';
  });
</script>
```

より発展した実装では、入力エラーHTMLをサーバーから返し、対象のフォーム領域へ差し替える。htmx 2.xでは `htmx:beforeSwap` で対象エラーレスポンスをスワップ対象として扱うよう設定する方法がある。

```
<script>
  document.body.addEventListener('htmx:beforeSwap', function (event) {
    if (event.detail.xhr.status === 422) {
      event.detail.shouldSwap = true;
      event.detail.isError = false;
    }
  });
</script>
```

注意: この設定だけでは表示先の構造は自動調整されない。フォーム用HTMLを返すなら、フォームを差し替えるターゲットを設計すること。また、すべての422を成功として扱うのではなく、対象リクエストに限定するのが望ましい。

入力中のサーバー検証

```
<label for="check-title">タイトル</label>
<input id="check-title" name="title"
       hx-get="api/validate"
       hx-trigger="input changed delay:300ms"
       hx-target="#validation">
<div id="validation" aria-live="polite"></div>
```

完成版Servletは 入力OK 等を返す。ここでの表示は**入力補助**であり、保存時のServlet検証を省略してよいわけではない。

Lesson 17 — 読み込み中を知らせる: hx-indicator

作るもの: 通信中だけ 保存中… を表示する。

```
<form hx-post="api/add"
      hx-target="#task-list"
      hx-indicator="#saving">
  <input name="title" required>
  <button type="submit">追加</button>
  <span id="saving" class="htmx-indicator" role="status">
    保存中…
  </span>
</form>
```

hx-indicator="#saving" は通信中に対象要素へ htmx-request クラスを付与する。htmx-indicator は表示切り替え用の既定クラス。

```
/* 独自の見た目にしたい場合 */
.htmx-indicator { display: none; }
.htmx-request.htmx-indicator,
.htmx-request .htmx-indicator { display: inline; }
```

既定のインジケータースタイルと併用するときは競合に注意する。本教材の完成版は独自CSSで表示を切り替える。厳格なCSPでは既定のインラインCSSを無効化して外部CSSへ移す方法もある。

二重送信対策

```
<form hx-post="api/add"
      hx-target="#task-list"
      hx-disabled-elt="find button[type='submit']">
  <input name="title" required>
  <button type="submit">追加</button>
</form>
```

`hx-disabled-elt` は送信中の対象要素を無効化する。ただし、サーバーへの重複書き込みを完全に防ぐ保証ではない。

演習: Networkでスロットリングし、保存中表示が出て消えることを確認する。

Lesson 18 — 追加後に別の領域も更新する: `HX-Trigger`

作るもの: タスク一覧が変わったら件数も変える。

```
<h2>タスク一覧（全 <span id="count"
    hx-get="api/count"
    hx-trigger="load, tasksChanged from:body">0</span> 件）</h2>
```

Servletが追加・削除後に以下のHTTPレスポンスヘッダーを付ける。

```
HX-Trigger: tasksChanged
```

```
response.setHeader("HX-Trigger", "tasksChanged");
```

順序:

1. `POST api/add`
2. Servletがタスクを追加し、一覧HTMLを返す
3. HTMXが `tasksChanged` イベントを発行
4. 件数表示要素が `tasksChanged from:body` でイベントを受信
5. `GET api/count`
6. 件数表示を更新

意味: ある操作をしたら「別の要素が自分の担当データを再取得する」という構成。画面の依存関係をJavaScriptで手作業更新する必要がない。

```
POST /api/add
   ├─ HTML断片 → #task-list
   └─ HX-Trigger: tasksChanged
                      └─ GET /api/count → #count
```

`HX-Trigger` は基本的に成功した非3xxレスポンスで利用する。通常のHTTPリダイレクトの3xxではHTMXのレスポンスヘッダーとして処理されない。

演習: DevToolsのNetworkで、追加操作がPOSTと件数取得GETの2通信になることを確認する。

Lesson 19 — 1回のレスポンスで複数領域を更新: `hx-swap-oob`

`HX-Trigger` の後続GETを省きたい場合、レスポンスに別領域用HTMLを同梱できる。これを **Out-of-Band（OOB）swap** と呼ぶ。

表示側

```
<ul id="task-list"></ul>
<div id="task-summary">全2件</div>
```

サーバーが返すHTML（例）

```
<li id="task-3">新しいタスク</li>
<li id="task-2">既存タスク</li>
<li id="task-1">既存タスク</li>

<div id="task-summary" hx-swap-oob="true">全3件</div>
```

リクエスト元が `hx-target="#task-list"` の場合、`<li>` 群は通常どおりリストの中へ入り、`#task-summary` は別の場所で要素ごと置換される。

注意: `hx-swap-oob="true"` を付けた断片は、ページ上の対応するIDと置換用タグ構造を一致させる。複雑な `<tr>`、`<td>`、`<li>` はHTMLの解析規則によって意図しない構造になるため、`<template>` を用いるパターン等を公式ドキュメントで確認する。

使い分け

方法	長所	向いている場面
`HX-Trigger`	各領域が独立して再取得	領域単位の分離、更新範囲が複雑
OOB	1応答で複数箇所を変更	件数・通知など関連断片を同時に返す

完成版は学習のため **`HX-Trigger`方式**。OOBはここで学ぶ追加パターンとして扱い、両方を同じ処理に重複適用しない。

Part 3 — HDAらしいページ遷移を作る

Lesson 20 — HTML全体から必要部分を取り出す: `hx-select`

これまではServletがHTML断片を返してきた。しかし、既存のMPAでは「同じURLで常にページ全体のHTMLを返す」設計もある。

例として、`GET /projects` が以下を返す場合を考える。

```
<!doctype html>
<html lang="ja">
```

```
<head><title>プロジェクト</title></head>
<body>
  <header>共通ナビゲーション</header>
  <main id="main-content">
    <h1>プロジェクト一覧</h1>
    <p>3件のプロジェクト</p>
  </main>
  <footer>共通フッター</footer>
</body>
</html>
```

別ページのHTMLの `main` 内側だけを取り込む例:

```
<button hx-get="projects"
        hx-select="#main-content"
        hx-target="#screen"
        hx-swap="innerHTML">
  プロジェクトを読み込む
</button>
<div id="screen"></div>
```

注意: この例は設計説明用。完成版Servletには `projects` エンドポイントがない。

`hx-select` は**受信HTMLのどの部分を選ぶか**、`hx-target` は**現在のページのどこを変えるか**、`hx-swap` は**どう挿入するか**を表す。上の例では選択された `<main>` 自体が `#screen` 内へ挿入される。`#screen` 自身を差し替えたいときは`outerHTML`と返却断片のID整合を検討する。

JSP/Servletでの選択肢

方法A: 同じURLでフルページ／断片を切り替える

```
通常GET /tasks       → ページ全体
HTMX GET /tasks       → <ul>内の<li>群
```

方法B: 通常のページURLと断片URLを分ける

```
GET /tasks           → ページ全体
GET /tasks/fragment  → 一覧HTML断片
```

どちらも有効。方法AはURLの一貫性、方法Bはサーバー処理の明確さが利点。重要なのは、**ブラウザーで直接開くURLがページ全体として正しく表示できること**。

Lesson 21 — 通常リンクを強化する: `hx-boost`

作るもの: リンクのURLを保持しつつ、全ページ再読込を減らす。

```
<nav hx-boost="true">
  <a href="tasks">タスク</a>
  <a href="projects">プロジェクト</a>
</nav>
```

htmx 2.x の `hx-boost="true"` は、対象となる通常リンク・フォームをAJAX通信へ切り替える。リンクの場合、既定では取得ページの内容を `<body>` の内側へ反映し、履歴も更新する。

重要な条件

- `href` が有効な実在URLであること
- JavaScriptを無効化しても同じリンクで遷移できるようにすること
- リンク先のサーバーが**完全なHTMLページ**を返すこと
- `<head>` にだけ置いたCSSやJSの更新が必要な設計では、文書全体更新との違いに注意すること

完成版の `api/tasks` は `<li>` しか返さない。したがって `href="api/tasks"` を `hx-boost` で強化して通常ページのように扱うのは誤り。実用版で `hx-boost` を使う場合は、ページ全体を返すURLを別途用意する。

メリット: 普通の `<a href="...">` から始められるため、段階的機能強化（progressive enhancement）と相性がよい。

Lesson 22 — 戻る・進む・共有可能URL: `hx-push-url`

作るもの: 検索条件をURLへ反映し、戻る操作を可能にする。

```
<form action="tasks" method="get"
      hx-get="tasks"
      hx-target="#content"
      hx-push-url="true">
  <label for="keyword">キーワード</label>
  <input id="keyword" name="q">
  <button type="submit">検索</button>
</form>
```

例えば `tasks?q=Java` のようなURLがアドレスバーに反映される。

守るべき条件: `hx-push-url="true"` で履歴へ積んだURLは、直接アクセスした場合にも**ページ全体として機能するHTML**を返す必要がある。`<li>` 断片しか返せないURLを履歴に積むと、更新・共有・履歴復元で表示が崩れる。

htmx 2.x は履歴スナップショットをブラウザー側で扱う。状況により履歴復元リクエストが発生することもある。サーバーはリクエストの種類を考慮して完全なページを返せるようにする。

```
HX-History-Restore-Request: true
```

扱う場合の考え方:

```
boolean historyRestore = "true".equals(
        request.getHeader("HX-History-Restore-Request"));
boolean htmxRequest = "true".equals(
        request.getHeader("HX-Request"));

if (historyRestore || !htmxRequest) {
    // フルページ用のビュー
} else {
    // HTML断片用のビュー
}
```

`hx-replace-url="true"` は現在の履歴項目のURLを書き換える用途。`hx-push-url` は新しい履歴を追加する用途。

発展課題: `/tasks?q=Java&status=done` をブラウザーで直接開いたとき、検索条件がフォームに復元されるSSR版を作る。

Lesson 23 — レスポンスヘッダーと画面遷移

画面状態変更の結果、URLを移動したい場合がある。

通常のMPA:

```
response.sendRedirect(request.getContextPath() + "/tasks");
```

HTMXリクエストに対してフルページ遷移を指示したい場合:

```
response.setHeader("HX-Redirect", request.getContextPath() + "/login");
response.setStatus(HttpServletResponse.SC_OK);
```

`HX-Redirect` はHTMXが受け取ってブラウザーを移動させる。`HX-Location` はページ全体の再読込を伴わないHTMX型ナビゲーションに利用できる。

レスポンスヘッダー一覧

ヘッダー	役割
`HX-Trigger`	イベントを発生させる
`HX-Trigger-After-Swap`	DOMスワップ後にイベント発生（v2）
`HX-Trigger-After-Settle`	反映処理完了後にイベント発生（v2）
`HX-Redirect`	フルページ遷移

ヘッダー	役割
HX-Location	HTMX型の画面遷移
HX-Push-Url	履歴へURL追加
HX-Replace-Url	現在の履歴URLを置換
HX-Refresh	ページ全体の再読込
HX-Retarget	レスポンスの対象変更
HX-Reswap	レスポンスのスワップ方式変更

注意: HX-Redirect のようなHTMX専用ヘッダーは**通常の3xxリダイレクトレスポンス上ではHTMXに処理されない**。認証切れ時の処理は通常遷移とHTMX通信の双方を試験する。

練習: API認証切れなら通常リクエストにはHTTP 303でログインページへ、HTMXからの通信には HX-Redirect でログインページへ導く方針を文章で設計する。実際の認証はPart 5で扱う。

Part 4 — 状態・イベント・高度なUI

Lesson 24 — JavaScriptを少量組み合わせる

HTMXの得意分野は「サーバーへ問い合わせてHTMLを差し替えること」。フォーカス移動、カメラ、チャート描画など、ブラウザー固有の処理にはJavaScriptが適する。

24.1 hx-on とカスタムイベント

```
<button hx-get="api/hello"
        hx-target="#message"
        hx-on::after-swap="console.log('HTMLを反映した')">
  読み込み
</button>
<div id="message"></div>
```

hx-on::after-swap は htmx:afterSwap の省略記法。this はイベントを受け取る要素。JavaScriptのイベント名はケバブケースに対応したHTMXの記法で扱う。

24.2 addEventListener 方式

```
<script>
  document.body.addEventListener('htmx:afterSwap', function (event) {
    console.log('更新先', event.detail.target);
  });
</script>
```

対象が少数で短い処理なら `hx-on`、複数画面へまたがる共通処理なら外部JavaScriptが整理しやすい。

24.3 やってはいけない混同

```
<!-- NG: サーバーから返った値をそのままJSとして実行 -->
<div hx-on::after-swap="eval(this.textContent)"></div>
```

外部入力・サーバー表示データをコードとして解釈しない。HTMXを使ってもXSSは発生し得る。

Lesson 25 — 遅延読み込み: `load` • `revealed` • `intersect`

作るもの: 画面が開いたときだけ「詳細」領域のHTMLを取得。

```
<div hx-get="api/clock"
     hx-trigger="load">
  準備中…
</div>
```

作るもの: スクロールで領域が見えるようになったとき取得。

```
<section hx-get="api/hello"
         hx-trigger="revealed">
  この位置に来たら読み込む
</section>
```

内部に独立したスクロール領域を持つ場合は `intersect` と適切なルート指定を使う。

```
<div hx-get="api/hello"
     hx-trigger="intersect once">
  表示領域に入るまで未取得
</div>
```

実用場面: プロフィール詳細の後読み、ダッシュボード内の重い統計、一覧末尾の追加取得など。

課題: ネットワークの遅延がある場合に、読み込み前のサイズと読み込み後のサイズが大きく変わらないよう仮表示領域の高さを確保する。

Lesson 26 — 無限スクロールの考え方

多数のデータを1回で全部返さず、ページ単位で追加する。

```
<ul id="records">
  <li>レコード1</li>
  <li>レコード2</li>
  <li id="more-trigger"
      hx-get="api/records?page=2"
      hx-trigger="revealed"
      hx-swap="outerHTML">
    続きを読み込み中…
  </li>
</ul>
```

2ページ目のサーバーレスポンス例

```
<li>レコード3</li>
<li>レコード4</li>
<li hx-get="api/records?page=3"
    hx-trigger="revealed"
    hx-swap="outerHTML">
  続きを読み込み中…
</li>
```

トリガー自身が次のレコード群と新トリガーに置き換わる。最後のページでは次トリガーを返さない。

この例は設計練習用。完成版Servletには `api/records` が存在しない。実装する場合は以下を満たす。

- サーバー側で1ページの件数上限を指定
- ページ番号、オフセット、カーソル等を受け取る
- 不正・過大なページ番号を検証
- 追加要求で同じレコードが重複しないようソート順を固定
- DBが更新される一覧ではカーソル方式なども検討

演習: 100件のタスクを想定し、1ページ20件としてリクエスト回数を設計する。

Lesson 27 — ダイアログ、モーダル、ファイルアップロード

27.1 モーダル

HTMXがモーダルを自動提供するわけではない。HTMLの `<dialog>` 等でUIを作り、**中身の取得だけHTMX**に任せる。

```
<button type="button"
        hx-get="api/hello"
        hx-target="#dialog-body"
        hx-on::after-swap="document.querySelector('#dialog').showModal()">
  詳細を表示
</button>
```

```
<dialog id="dialog">
  <div id="dialog-body"></div>
  <form method="dialog"><button>閉じる</button></form>
</dialog>
```

表示・閉じる・フォーカスなどはブラウザーの標準機能を活用する。確認やフォーム検証の挙動も試験すること。

27.2 ファイルアップロード

```
<form hx-post="api/upload"
      hx-encoding="multipart/form-data"
      hx-target="#upload-result">
  <label>画像ファイル
    <input type="file" name="image" accept="image/*" required>
  </label>
  <button type="submit">アップロード</button>
</form>
<div id="upload-result"></div>
```

Servlet側では `@MultipartConfig`、`request.getPart("image")` などで受け取る。上記の `api/upload` は**完成版にない追加課題用URL**。

アップロードでは以下を必須で設計する。

- ファイルサイズ制限
- MIMEタイプや実データの検査
- ファイル名のサニタイズと保存先の固定
- 公開権限の確認
- マルウェア対策や画像再エンコードの検討
- 認証・CSRF保護

27.3 SSEとWebSocket

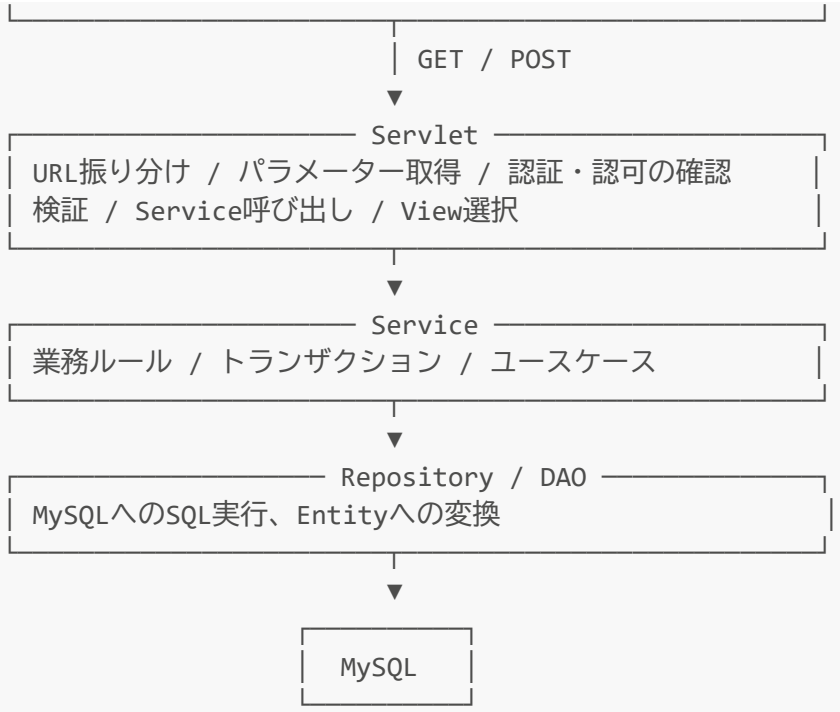
htmxはSSE/WebSocketを拡張で利用できる。チャットやリアルタイム通知で検討する。**通常のフォームやCRUDでは必要になるまで追加しない**。

Part 5 — Servletと接続するための設計

Lesson 28 — HTMXとServletの責務を分ける

授業で作る小規模アプリから、実際のWebアプリへ移行するときの役割を整理する。

```
┌──────────────────── ブラウザー ────────────────────┐
│ HTML / CSS / HTMX / 必要最小限のJavaScript          │
│ ① 操作  ② HTTP送信  ⑤ HTML断片の差し替え            │
```



方針

層	原則
Browser	HTMXは通信と描画更新を担当。永続データの正本にならない
Servlet/Controller	HTTPに関する処理と画面選択。複雑なSQLを書き込まない
Service	入力後の業務ルールとトランザクション
Repository/DAO	DBアクセスを集約
JSP/View	サーバー側でHTMLを生成。業務ロジックを記述しない

本教材の `TaskServlet` は**動く教材を1ファイルで確認できることを優先**し、保存とHTML生成を同じクラスに置いている。本番では責務分離する。

Lesson 29 — ServletはHTML断片とページをどう返すか

29.1 学習版: JavaからHTMLを直接返す

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        response.getWriter().write("<p>こんにちは</p>");
    }
}
```

HTMX側:

```
<button hx-get="hello" hx-target="#output">取得</button>
<div id="output"></div>
```

構造の理解には最短。ただしHTMLが複雑になるとJava文字列を大量に組み立てる方法は読みにくい。

29.2 実用版: ServletからJSPへforward

```
@WebServlet("/tasks")
public class TasksServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        // serviceから取得済みと仮定。ダミーデータの例
        List<String> titles = List.of("HTML復習", "HTMX学習");
        request.setAttribute("titles", titles);

        response.setContentType("text/html;charset=UTF-8");
        request.getRequestDispatcher(
                "/WEB-INF/views/fragments/task-list.jsp")
                .forward(request, response);
    }
}
```

`task-list.jsp`:

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<c:forEach var="title" items="${titles}">
  <li><c:out value="${title}" /></li>
</c:forEach>
```

このJSP断片は `<ul>` の内側に配置する想定。

JSTLを使う場合のMaven依存追加例

```
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
  <version>3.0.0</version>
</dependency>
<dependency>
  <groupId>org.glassfish.web</groupId>
```

```
    <artifactId>jakarta.servlet.jsp.jstl</artifactId>
    <version>3.0.1</version>
  </dependency>
```

Tomcat 11/Jakarta系では `jakarta.tags.core` を使用する。古い `http://java.sun.com/jsp/jstl/core` と混在させない。

29.3 full/fragment分岐

```
boolean htmx = "true".equals(request.getHeader("HX-Request"));
boolean restore = "true".equals(
        request.getHeader("HX-History-Restore-Request"));

String view = htmx && !restore
        ? "/WEB-INF/views/fragments/task-list.jsp"
        : "/WEB-INF/views/tasks/index.jsp";

response.setHeader("Vary", "HX-Request, HX-History-Restore-Request");
request.getRequestDispatcher(view).forward(request, response);
```

同じURLで別表現を返す場合、キャッシュがフルページHTMLと断片HTMLを混同しないよう注意する。`Vary` はそのための基本策の一つ。個人情報がある場合は、追加で適切な `Cache-Control` を設定する。

注意: 通常ブラウザー遷移、HTMXリクエスト、履歴復元の各ケースを必ず動作確認する。

Lesson 30 — JSPでHTMLを安全に生成する

JSPにDBの値を埋め込むとき、**HTMLエスケープ**が必須。

危険な例

```
<!-- ユーザー入力値を直接HTML出力する書き方は避ける -->
<%= request.getParameter("title") %>
```

`<script>...</script>` 等が入力されるとXSSの原因になる。

基本形

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<span><c:out value="${task.title}" /></span>
```

属性に出す値も適切にエスケープする。

```
<input name="title" value="<c:out value='${task.title}' />">
```

一般に、URL・JavaScript・CSS・HTML属性など**出力先コンテキストによって適切なエンコード処理は異なる**。`<c:out>` は基本的なHTML/XML文字エスケープの手段であり、JavaScript文字列へ直接埋める安全性まで保証するものではない。

原則

- サーバーはユーザー入力を安全にエスケープしてHTMLへ埋める
- サーバーは不正なデータを検証で拒否する
- `innerHTML` に任意入力を直接組み立てて挿入しない
- `hx-vals='js:...'` 等の動的評価へ外部入力を混ぜない
- ユーザー作成HTMLを許可するなら、専用サニタイザーと許可ルールを設計する

HTMXを使ってもXSS対策の責務はなくならない。

Lesson 31 — JavaScriptなしでも使える構成を目指す

完成版タスクラボはHTMXの練習に焦点を当て、JavaScriptが無効だと一覧取得や操作ができない。そのままでは完全な**プログレッシブエンハンスメント（段階的機能強化）**構成ではない。

実用版は通常のHTMLを基盤にして、その上からHTMXを強化手段として追加する。

```
<!-- JavaScriptなし: 普通のPOSTフォームとして動く -->
<!-- JavaScriptあり: HTMXにより一覧だけ更新する -->
<form action="tasks" method="post"
      hx-post="tasks"
      hx-target="#task-list">
  <label>タイトル
    <input name="title" required>
  </label>
  <button type="submit">追加</button>
</form>
```

Servlet側の処理方針:

```
POST /tasks
  入力検証
  DBに保存
  ├─ HX-Request: true
  │      → 200 + HTML断片
  └─ 通常リクエスト
         → 303 See Other → GET /tasks
```

```
if ("true".equals(request.getHeader("HX-Request"))) {
    request.setAttribute("tasks", service.findAll());
    request.getRequestDispatcher("/WEB-INF/views/fragments/task-list.jsp")
           .forward(request, response);
} else {
    response.setStatus(HttpServletResponse.SC_SEE_OTHER);
    response.setHeader("Location", request.getContextPath() + "/tasks");
}
```

通常フォームのPOST後にGETへ誘導する方式は **Post/Redirect/Get（PRG）**。更新・再読込による意図しない再POSTを減らす。

削除も同様に、通常フォームへ `action` と `method` を持たせ、その上に `hx-post` を付けられる。

```
<form method="post" action="tasks/delete">
  <input type="hidden" name="id" value="1">
  <button type="submit" hx-post="tasks/delete"
          hx-include="closest form"
          hx-target="#task-list">削除</button>
</form>
```

これは設計例であり、完成版ServletのURLとは異なる。本番ではHTMLとサーバー側のルートを一致させる。

到達基準: 直接URLを開く、再読み込み、戻る、JavaScript無効、HTMX有効の各状態でデータを壊さず動ける。

Lesson 32 — CSRF・認証・認可を理解する

32.1 HTMXに認証は含まれない

ログイン中のCookieは同一オリジンのリクエストへ通常送られる。したがって、Servletの `HttpSession` を使う構成と共存できる。

```
HttpSession session = request.getSession(false);
if (session == null || session.getAttribute("userId") == null) {
    response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
    return;
}
```

これだけでは十分ではない。**各操作対象の所有者や権限**をサーバー側で確認する。

```
POST /tasks/delete?id=50
→ ログインしているか
```

```
→ このユーザーがID=50のタスクにアクセス可能か
→ 可能なら削除
```

`hx-include` で `userId` を送る方式に認可を依存しない。ユーザーIDの正本はサーバーセッションや正当な認証情報に基づく。

32.2 CSRF

Cookie認証を使う更新操作ではCSRF対策を検討する。**HTMXが自動でCSRFを無効化してくれるわけではない**。

防御例:

1. ログインセッションに紐づく予測困難なCSRFトークンをサーバーで生成する
2. ページ内のフォームにhidden input等で埋め込む
3. POST時に送られたトークンをサーバー側で検証する
4. `SameSite` Cookie、`Origin`/`Referer` の検証等を補助策として使う

```
<!-- token値はサーバーで生成・HTML属性用にエスケープする -->
<form action="tasks" method="post" hx-post="tasks">
  <input type="hidden" name="csrfToken" value="SERVER_TOKEN">
  <input name="title" required>
  <button type="submit">追加</button>
</form>
```

この方式ならフォーム内のトークンをHTMXも送る。`hx-headers` にCSRFトークンを設定する方式もある。トークンの検証とセッションへの紐づけは**必ずサーバー側**で実施する。

32.3 認証切れとHTTPステータス

```
Ajax要求 → セッション切れ → 401 + 明示的な再ログインUI
                                または HX-Redirect
```

通常ページへログインHTMLを返して断片部分に埋め込んでしまう挙動は避ける。

32.4 その他の対策

- HTTPSの強制
- セッションCookieの `Secure`・`HttpOnly`・`SameSite`
- XSS対策とコンテンツセキュリティポリシー（CSP）
- SQLインジェクション防止
- IDを変更して他人のデータにアクセスできないか確認（IDOR対策）
- エラーページにスタックトレースを出さない
- リクエスト頻度制限、サイズ制限、監査ログ

Lesson 33 — メモリ保存からMySQLへ移行する

本教材のServletは `ConcurrentHashMap<Long, Task>` を使用する。これはアプリ再起動で消え、複数サーバー間で共有されない。

MySQLへ移行する際の最小例:

```
CREATE TABLE tasks (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT NOT NULL,
    title VARCHAR(80) NOT NULL,
    done BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_tasks_user_id (user_id)
);
```

実際には `user_id` の外部キーやユーザーテーブルなど、システム全体の設計に合わせる。

Repository例:

```
public List<Task> findByUserId(long userId) throws SQLException {
    String sql = "SELECT id, title, done FROM tasks "
               + "WHERE user_id = ? ORDER BY id DESC";
    try (Connection connection = dataSource.getConnection();
         PreparedStatement statement = connection.prepareStatement(sql)) {
        statement.setLong(1, userId);
        try (ResultSet rs = statement.executeQuery()) {
            List<Task> result = new ArrayList<>();
            while (rs.next()) {
                result.add(new Task(
                    rs.getLong("id"),
                    rs.getString("title"),
                    rs.getBoolean("done")
                ));
            }
            return result;
        }
    }
}
```

`Task`、`dataSource`、必要なimportは各層の実装で定義する。この断片はDAOパターンの説明であり、単独ではコンパイル対象にならない。

DB化のチェックリスト

- 接続情報をソースへ直書きしない
- JDBCの `PreparedStatement` で値をバインドする
- 更新後に影響行数と所有者条件を確認する
- 複数の関連更新ではトランザクションを使う

- ページング用の検索条件とインデックスを検討する
- 接続プール（DataSource/JNDI等）を使う
- 1件更新ごとに無関係な大量HTMLを再生成しないか測定する

HTMXが変わってもService/DAOの主要部分は原則そのまま利用できる。**フロント側のAPI契約がJSONではなくHTMLである**、という違いに集中する。

Part 6 — デバッグ・品質・習熟

Lesson 34 — DevToolsでHTMXの通信を追う

HTMX開発では**ブラウザーのNetworkが最重要**。

手順:

1. 開発者ツールを開く
2. **Network** を選ぶ
3. Fetch/XHR等に絞る
4. タスクを追加する
5. `api/add` のリクエストを選ぶ
6. Headers / Payload / Response / Timing を見る

確認するポイント:

見る場所	確認事項
Request URL	意図したコンテキストパスか
Request Method	GETかPOSTか
Query String	`id`, `q`, `status` があるか
Form Data	`title` が送られているか
Request Headers	`HX-Request: true` があるか
Status Code	`200`, `204`, `422`, `404`, `500` 等
Response	JSONではなく期待したHTMLか
Response Headers	`HX-Trigger` が存在するか
Elements	`#task-list` が書き換わったか

34.1 HTMXイベントのログ

開発中にConsoleへ次を入力する。

```
htmx.logAll();
```

通信・スワップ関連イベントを追える。常時の詳細ログは膨大なので、本番では必要に応じて抑える。

34.2 よくある不具合

症状	疑う原因	対処
ボタンが反応しない	HTMXが未読み込み	NetworkでCDN、Consoleで `typeof htmx`
404	URL、コンテキストパス違い	Request URLとServletマッピング確認
一覧が更新されない	`hx-target` が不一致	DOM上のIDを確認
追加できない	`name="title"` がない	Payloadを確認
文字化け	UTF-8設定不足	HTML/Servlet/DB接続の文字コード確認
エラーが表示されない	v2は4xx/5xxを既定でスワップしない	`responseError`等で扱う
編集ボタンが消える	`outerHTML` でタグやIDを消した	レスポンス構造を確認
検索結果が古い	通信競合	`hx-sync` とサーバー動作を確認
追加できるが更新すると消える	ブラウザー側だけ変更している	サーバーの保存処理確認
保存後に意図しないページが表示	リダイレクト・フルページ/断片の不一致	HTTP応答とターゲット確認

Lesson 35 — HTMXアプリのテストを設計する

35.1 Servlet単体・結合テスト

```
GET /api/tasks
→ status 200
→ Content-Type text/html;charset=UTF-8
→ <li>群を返す

POST /api/add (title=Java)
→ status 200
→ 新タスクが一覧に存在する
→ HX-Trigger = tasksChanged

POST /api/add (title=空)
→ status 422
→ データが増えない

POST /api/delete?id=存在しないID
```

```
→ status 404
→ 既存データに影響しない
```

35.2 ブラウザー操作テスト

- 初回表示で一覧が取得される
- タスクを追加すると表示と件数が変わる
- 削除確認でキャンセルすると消えない
- 完了状態を変更すると見た目とサーバーの値が一致する
- 編集・取消・保存がそれぞれ正しく動く
- 検索語を変えても古い検索結果が最終表示に残らない
- 絞り込み中の変更後も条件を保持する
- キーボードだけでボタンとフォームを操作できる
- ネットワーク切断時に障害を通知する
- HTML注入を試してもスクリプトが実行されない
- `file://` ではなくTomcatのURLからアクセスできる

35.3 基本的なアクセシビリティ

HTMLの意味を守ればHTMXでも基本の対策は同じ。

- クリック可能なものは `<button>`、遷移は `<a>` を使う
- `<input>` と `<label>` を関連付ける
- フォーカス枠を消さない
- 検索結果と通知に必要に応じて `aria-live` を使う
- 大きくHTMLを交換した後のフォーカス位置を確認する
- モーダルの開閉時はキーボード操作・フォーカス制御を確認する
- 色だけで完了・エラーを区別しない

注意: `aria-live` を大量のDOM更新領域へ一律に付けると、読み上げが冗長になる。更新範囲や `aria-relevant` を調整し、実際のスクリーンリーダーで試験する。

Lesson 36 — HTMX 2.xと4.0を混在させない

2026年10月時点でHTMX 4.0は公開済み。ただし2.x系列とは動作・構文の違いがある。本教材は**2.0.11で統一**。

観点	この教材の2.x	4.0での主な変更
4xx/5xx応答	既定では通常スワップしない	原則スワップする（204/304等を除く）
属性の継承	一部属性が暗黙的に継承される	継承方針が変更され、明示的な指定が重要
History	スナップショットキャッシュを利用する方式	既定ではサーバーから再取得する方式
`HX-Trigger`	Before/After系が分かれる	レスポンス後のイベント体系が変更

観点	この教材の2.x	4.0での主な変更
`hx-boost`	v2の既定動作を前提	v4の新たな既定動作・修飾子を確認
`hx-delete`	v2のパラメーター規則	フォーム値の扱いが変更

重要: 4.0のサイトにあるサンプルの属性やイベント処理を2.0.11のコードへそのまま貼らない。バージョンを固定し、該当する公式ドキュメントを参照する。

参照先:

- htmx 2.x Documentation
- htmx 2.x Reference
- htmx 4 Documentation
- What's New in htmx 4

Lesson 37 — ミニ開発課題で自力実装する

ここからは完成版を見ずに取り組む。

課題A: タスクの「重要」フラグ

要件

1. タスクに `important` 真偽値を追加する
2. 行に「重要／解除」ボタンを配置する
3. `POST api/important?id=...` を作る
4. サーバー側の状態を更新する
5. 更新後の一覧HTMLを返す
6. 重要タスクはCSSで強調する

使う技術: `hx-post`, `hx-target`, 断片生成, Servletの `doPost()`。

課題B: 期限付きタスク

要件

1. フォームに `<input type="date" name="dueDate">` を追加
2. Servletで日付を検証
3. タスク一覧に期限を表示
4. 期限の近い順に並べ替える
5. 不正な日付はHTTP 422

使う技術: フォームパラメーター、サーバー検証、HTMLエスケープ。

課題C: ページング

要件

1. サーバーが1回に20件だけ返す
2. 次ページをクリック、またはスクロールで取得

3. 重複・欠損しない並び順を決める
4. 検索条件を次ページの要求へ引き継ぐ
5. データが尽きたら次トリガーを出さない

使う技術: `hx-get`, `hx-swap`, `hx-trigger`, `hx-include`。

課題D: 最もHDAらしい「複数画面」

1. `/tasks`、`/projects` で**ページ全体**を返す
2. 一覧断片用JSPを再利用する
3. 通常の `<a href="...">` で遷移できる
4. `hx-boost` で遷移を強化する
5. 直接URLアクセス、リロード、戻るが正しく機能する
6. 認証・認可とCSRF保護を追加する

達成すれば、教材の目標であるServletベースのHDAを構成する基礎が完成。

まとめ — HDA開発の設計原則

1. **HTMLをAPIの表現として扱う**: サーバーが次の操作に必要なHTMLを返す
2. **サーバーを状態の正本にする**: DOMは表示であり、永続データではない
3. **操作→HTTP→HTML断片→スワップ**を基本単位にする
4. **GETは参照、POST等は状態変更**の原則を守る
5. **`hx-target` と `hx-swap` は必ず対で設計**する
6. **バリデーション・認証・認可をServlet/Service側で強制**する
7. **フルページと断片の責務を明確にする**
8. **通常URLを壊さず段階的に強化**する
9. **必要なJavaScriptは少量、明示的に追加**する
10. **Networkで通信・HTML・HTTPステータスを確認**する

属性早見表（htmx 2.x）

属性	用途	最小例
`hx-get`	GET	`hx-get="api/tasks"`
`hx-post`	POST	`hx-post="api/add"`
`hx-put`	PUT	`hx-put="api/tasks/1"`
`hx-patch`	PATCH	`hx-patch="api/tasks/1"`
`hx-delete`	DELETE	`hx-delete="api/tasks/1"`
`hx-target`	更新対象	`hx-target="#result"`
`hx-swap`	反映方法	`hx-swap="outerHTML"`

属性	用途	最小例
hx-trigger	起動条件	hx-trigger="change"
hx-include	追加パラメーター	hx-include="#search"
hx-vals	値の追加	hx-vals='{"level":1}'
hx-params	パラメーターの制限	hx-params="not debug"
hx-confirm	確認ダイアログ	hx-confirm="削除する？"
hx-indicator	通信中表示	hx-indicator="#wait"
hx-disabled-elt	通信中の無効化	hx-disabled-elt="this"
hx-sync	競合・連続通信制御	hx-sync="this:replace"
hx-select	レスポンス内の部分選択	hx-select="#main"
hx-swap-oob	別の要素も更新	hx-swap-oob="true"
hx-boost	普通のリンク／フォーム強化	hx-boost="true"
hx-push-url	履歴へURL追加	hx-push-url="true"
hx-replace-url	履歴URLを置換	hx-replace-url="true"
hx-on::*	HTMXイベントへの処理	hx-on::after-swap="..."
hx-encoding	送信形式指定	hx-encoding="multipart/form-data"

hx-put、hx-patch、hx-delete の例は属性構文のみ。**対応URLとHTTPメソッドをServletに実装しなければ動作しない**。

付録 — コピーして動かせる完成ソースコード

> **PDF版の読みやすさについて**: 付録B・Cの長いソースコードは、**ページをまたがない長さのコードブロック**に分割して掲載する。各「続き」ブロックは直前のコードの継続部分。該当するファイルへ**順番どおりにコード部分だけを連結**する。ラベルの文章はコピーしない。コード自体は省略・変更していない。

ここからはPart 0で案内した完成版。**1つのMarkdownファイルに、必要な3ファイルの全文**を格納している。実際の開発環境では見出しに記載したパスへ分けて保存する。

付録A. 既存 multi-management-system/pom.xml の設定

A.1 必須: 既存POMへ不足設定を追加する

既存POMを全文上書きしない。以下は追記する要素の例。<project> の下の既存の <properties>、<dependencies>、<build> に**中身だけを統合**する。既に同じ依存やプラグインがある場合、重複登録せず値を調整する。

<project> 直下に war を指定する。

```
  <packaging>war</packaging>
```

`<properties>` の内部:

```
  <maven.compiler.release>25</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

`<dependencies>` の内部:

```
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.1.0</version>
    <scope>provided</scope>
  </dependency>
```

`<build>` の内部（`plugins` がなければ作る）:

```
  <finalName>multi-management-system</finalName>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.15.0</version>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-war-plugin</artifactId>
      <version>3.4.0</version>
    </plugin>
  </plugins>
```

注意: これはXML断片の例であり、別々の `<build>` 等を新設して貼り付ける意味ではない。既存のプラグインや依存関係を維持すること。実際のPOMの構造が異なる場合は、既存の構造を優先する。

A.2 参考: 上記を含む最小POMの全体形

以下は必要最小限の例であり、**既存プロジェクトのPOMと入れ替えるためのものではない**。

```
  <project xmlns="http://maven.apache.org/POM/4.0.0"
           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
           xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  https://maven.apache.org/xsd/maven-4.0.0.xsd">
      <modelVersion>4.0.0</modelVersion>
```

```
    <groupId>team.multi.project</groupId>
    <artifactId>multi-management-system</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>

    <properties>
        <maven.compiler.release>25</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>jakarta.servlet</groupId>
            <artifactId>jakarta.servlet-api</artifactId>
            <version>6.1.0</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <build>
        <finalName>multi-management-system</finalName>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.15.0</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-war-plugin</artifactId>
                <version>3.4.0</version>
            </plugin>
        </plugins>
    </build>
</project>
```

付録B. src/main/java/team/multi/project/tutorial/TaskServlet.java

```
package team.multi.project.tutorial;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Comparator;
```

```
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

@WebServlet("/htmx-lab/api/*")
public final class TaskServlet extends HttpServlet {
    private record Task(long id, String title, boolean done) {}

    private final Map<Long, Task> tasks = new ConcurrentHashMap<>();
```

TaskServlet.java — 続き（2/13）

```
    private final AtomicLong ids = new AtomicLong();

    @Override
    public void init() throws ServletException {
        create("HTMLの仕組みを復習する");
        create("hx-getを使ってみる");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws IOException {
        prepare(req, res);
        String path = req.getPathInfo();
        if (path == null) path = "/";

        switch (path) {
            case "/hello" -> html(res, "<p>ServletからHTMLが届いた！</p>");
            case "/clock" -> html(res, "<time>" + escape(

LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss")))
                    + "</time>");
            case "/tasks" -> html(res, rows(req));
            case "/count" -> html(res, String.valueOf(tasks.size()));
```

TaskServlet.java — 続き（3/13）

```
            case "/edit" -> {
                Task task = getTask(req);
                if (task == null) {
                    notFound(res);
                } else {
                    html(res, editRow(task));
                }
            }
            case "/view" -> {
                Task task = getTask(req);
```

```
                if (task == null) {
                    notFound(res);
                } else {
                    html(res, row(task));
                }
            }
            case "/preview" -> {
                String title = text(req.getParameter("title"));
                html(res, "<p>プレビュー: " + escape(title) + "</p>");
            }
```

TaskServlet.java — 続き（4/13）

```
            case "/validate" -> {
                String title = text(req.getParameter("title")).trim();
                String message = title.isEmpty()
                        ? "タイトルを入力"
                        : title.length() > 80 ? "80文字以内" : "入力OK";
                html(res, "<small>" + escape(message) + "</small>");
            }
            case "/empty" -> res.setStatus(HttpServletResponse.SC_NO_CONTENT);
            case "/failure" -> {
                res.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
                html(res, "<p>サーバーでエラーが発生</p>");
            }
            default -> notFound(res);
        }
    }
```

TaskServlet.java — 続き（5/13）

```
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws IOException {
        prepare(req, res);
        String path = req.getPathInfo();
        if (path == null) path = "/";

        switch (path) {
            case "/add" -> {
                String title = text(req.getParameter("title")).trim();
                if (!valid(title)) {
                    invalid(res);
                    return;
                }
                create(title);
                changed(res);
```

```
                html(res, rows(req));
            }
```

TaskServlet.java — 続き（6/13）

```
            case "/toggle" -> {
                Long id = parseId(req);
                if (id == null || tasks.computeIfPresent(id,
                        (key, value) -> new Task(value.id(), value.title(),
!value.done())) == null) {
                    notFound(res);
                    return;
                }
                changed(res);
                html(res, rows(req));
            }
            case "/delete" -> {
                Long id = parseId(req);
                if (id == null || tasks.remove(id) == null) {
                    notFound(res);
                    return;
                }
                changed(res);
                html(res, rows(req));
            }
```

TaskServlet.java — 続き（7/13）

```
            case "/save" -> {
                Long id = parseId(req);
                String title = text(req.getParameter("title")).trim();
                if (!valid(title)) {
                    invalid(res);
                    return;
                }
                if (id == null || tasks.computeIfPresent(id,
                        (key, value) -> new Task(value.id(), title, value.done()))
== null) {
                    notFound(res);
                    return;
                }
                changed(res);
                html(res, rows(req));
            }
            default -> notFound(res);
        }
    }

    private void create(String title) {
```

TaskServlet.java — 続き（8/13）

```
        long id = ids.incrementAndGet();
        tasks.put(id, new Task(id, title, false));
    }

    private String rows(HttpServletRequest req) {
        String q =
text(req.getParameter("q")).trim().toLowerCase(java.util.Locale.ROOT);
        String status = text(req.getParameter("status"));
        List<Task> result = tasks.values().stream()
                .filter(task ->
task.title().toLowerCase(java.util.Locale.ROOT).contains(q))
                .filter(task -> switch (status) {
                    case "open" -> !task.done();
                    case "done" -> task.done();
                    default -> true;
                })
                .sorted(Comparator.comparingLong(Task::id).reversed())
                .toList();
        if (result.isEmpty()) return "<li class=\"empty\">該当するタスクなし</li>";
        StringBuilder out = new StringBuilder();
        result.forEach(task -> out.append(row(task)));
        return out.toString();
    }
```

TaskServlet.java — 続き（9/13）

```
    private String row(Task task) {
        return """
                <li id="task-%d" class="task-row%s">
                  <span class="task-title">%s</span>
                  <span class="state">%s</span>
                  <button type="button" hx-post="api/toggle?id=%d"
                          hx-include="#search, #status"
                          hx-target="#task-list">%s</button>
                  <button type="button" hx-get="api/edit?id=%d"
                          hx-target="closest li" hx-swap="outerHTML">編集</button>
                  <button type="button" hx-post="api/delete?id=%d"
                          hx-include="#search, #status"
                          hx-target="#task-list"
                          hx-confirm="このタスクを削除する？">削除</button>
                </li>
                """.formatted(
                task.id(), task.done() ? " done" : "",
                escape(task.title()), task.done() ? "完了" : "未完了",
                task.id(), task.done() ? "戻す" : "完了にする", task.id(),
task.id());
    }

    private String editRow(Task task) {
```

TaskServlet.java — 続き（10/13）

```
        return """
                <li id="task-%d" class="task-row">
                  <form hx-post="api/save?id=%d" hx-target="#task-list"
                        hx-include="#search, #status">
                    <label>タイトル
                      <input name="title" value="%s" maxlength="80" required
autofocus>
                    </label>
                    <button type="submit">保存</button>
                    <button type="button" hx-get="api/view?id=%d"
                            hx-target="closest li" hx-swap="outerHTML">取消
</button>
                  </form>
                </li>
                """.formatted(task.id(), task.id(), escape(task.title()),
task.id());
    }

    private Task getTask(HttpServletRequest req) {
        Long id = parseId(req);
        return id == null ? null : tasks.get(id);
    }
```

```
    private Long parseId(HttpServletRequest req) {
```

TaskServlet.java — 続き（11/13）

```
        try {
            return Long.valueOf(req.getParameter("id"));
        } catch (NumberFormatException | NullPointerException ex) {
            return null;
        }
    }

    private boolean valid(String title) {
        return !title.isBlank() && title.length() <= 80;
    }

    private String text(String value) {
        return value == null ? "" : value;
    }

    private String escape(String input) {
        return input.replace("&", "&amp;")
                .replace("<", "&lt;")
                .replace(">", "&gt;")
                .replace("\"", "&quot;")
                .replace("'", "&#39;");
    }
```

TaskServlet.java — 続き（12/13）

```
    private void prepare(HttpServletRequest req, HttpServletResponse res)
            throws IOException {
        req.setCharacterEncoding("UTF-8");
        res.setCharacterEncoding("UTF-8");
        res.setContentType("text/html;charset=UTF-8");
        res.setHeader("Cache-Control", "no-store");
    }

    private void html(HttpServletResponse res, String fragment) throws IOException
{
        res.getWriter().write(fragment);
    }

    private void changed(HttpServletResponse res) {
        res.setHeader("HX-Trigger", "tasksChanged");
    }

    private void invalid(HttpServletResponse res) throws IOException {
```

```
            res.setStatus(422);
            html(res, "<p>タイトルは1〜80文字で入力</p>");
        }
```

TaskServlet.java — 続き（13/13）

```
    private void notFound(HttpServletResponse res) throws IOException {
        res.setStatus(HttpServletResponse.SC_NOT_FOUND);
        html(res, "<p>対象が見つからない</p>");
    }
}
```

付録C. src/main/webapp/htmx-lab/index.html

```
<!doctype html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>HTMX タスクラボ</title>
  <script src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.11/dist/htmx.min.js"
          integrity="sha384-
2OatzQy1H+Zd/IIrjr1TcuDGqLXeHhbooAyJY1KdQMKnr4LZ22k31GBLdYKHmVjg"
          crossorigin="anonymous" defer></script>
  <style>
    :root { color-scheme: light dark; font-family: system-ui, sans-serif; }
    body { max-width: 780px; margin: 2rem auto; padding: 0 1rem; line-height: 1.6;
}
    input, button, select { font: inherit; padding: .4rem .6rem; }
    button { cursor: pointer; }
    button:focus-visible, input:focus-visible, select:focus-visible {
      outline: 3px solid #5a99d9; outline-offset: 2px;
    }
```

index.html — 続き（2/5）

```
    .bar { display: flex; flex-wrap: wrap; gap: .6rem; margin: 1rem 0; }
    .bar input { flex: 1; min-width: 10rem; }
    #task-list { list-style: none; padding: 0; }
    .task-row { display: flex; align-items: center; flex-wrap: wrap;
      gap: .6rem; padding: .75rem; border-bottom: 1px solid #9995; }
    .task-row form { display: flex; gap: .6rem; align-items: center; flex-wrap:
wrap; }
    .task-title { flex: 1; min-width: 9rem; overflow-wrap: anywhere; }
    .done .task-title { text-decoration: line-through; opacity: .65; }
    .state { font-size: .85em; opacity: .7; }
```

```
    .htmx-indicator { display: none; }
    .htmx-request .htmx-indicator, .htmx-request.htmx-indicator { display: inline;
}
    #feedback { min-height: 1.5em; }
  </style>
</head>
<body>
  <main>
    <h1>HTMX タスクラボ</h1>
    <p>Servletが状態を保持し、HTML断片を返すアプリ</p>
```

index.html — 続き（3/5）

```
    <section aria-labelledby="first-heading">
      <h2 id="first-heading">まずは通信</h2>
      <button type="button" hx-get="api/hello" hx-target="#hello">挨拶を取得
</button>
      <button type="button" hx-get="api/clock" hx-target="#clock">時刻を取得
</button>
      <div id="hello" aria-live="polite"></div>
      <div id="clock" aria-live="polite"></div>
    </section>

    <section aria-labelledby="task-heading">
      <h2 id="task-heading">タスク一覧 <small>(全 <span id="count"
        hx-get="api/count" hx-trigger="load, tasksChanged from:body">0</span> 件)
</small></h2>

      <form class="bar" hx-post="api/add" hx-target="#task-list"
            hx-include="#search, #status"
            hx-on::after-request="if (event.detail.successful) this.reset()"
            hx-indicator="#saving">
        <label for="title">新しいタスク</label>
        <input id="title" name="title" required maxlength="80"
               placeholder="例: 設計図を描く">
        <button type="submit">追加</button>
        <span id="saving" class="htmx-indicator" role="status">保存中…</span>
      </form>
```

index.html — 続き（4/5）

```
      <div class="bar">
        <label for="search">検索</label>
        <input id="search" name="q" type="search"
               placeholder="キーワード"
               hx-get="api/tasks" hx-trigger="input changed delay:350ms, search"
               hx-include="#status" hx-target="#task-list"
```

```
                hx-sync="#task-list:replace" hx-indicator="#searching">
          <label for="status">状態</label>
          <select id="status" name="status"
                  hx-get="api/tasks" hx-trigger="change"
                  hx-include="#search" hx-target="#task-list"
                  hx-sync="#task-list:replace">
            <option value="all">すべて</option>
            <option value="open">未完了</option>
            <option value="done">完了</option>
          </select>
          <span id="searching" class="htmx-indicator" role="status">検索中…</span>
        </div>
```

index.html — 続き（5/5）

```
        <ul id="task-list" aria-live="polite" aria-relevant="additions removals"
            hx-get="api/tasks" hx-trigger="load" hx-include="#search, #status">
          <li>読み込み中…</li>
        </ul>
        <p id="feedback" role="status" aria-live="polite"></p>
      </section>
    </main>
    <script>
      document.body.addEventListener('htmx:responseError', function (event) {
        document.querySelector('#feedback').textContent =
          '処理に失敗（HTTP ' + event.detail.xhr.status + '）';
      });
      document.body.addEventListener('htmx:sendError', function () {
        document.querySelector('#feedback').textContent = '通信できない';
      });
      document.body.addEventListener('htmx:afterRequest', function (event) {
        if (event.detail.successful) {
          document.querySelector('#feedback').textContent = '';
        }
      });
    </script>
  </body>
  </html>
```

付録D — 完成版の動作確認（VS Code + Tomcat 11）

必須: Lesson 02で `study/htmx-hda` に切り替えてから作業する。`main` 上でチュートリアル用ファイルを作成しない。

1. **既存の** `multi-management-system/pom.xml` を付録Aに従って統合編集する。ほかの依存関係やプロジェクト設定を消さない
2. 付録Bの `TaskServlet.java` を `src/main/java/team/multi/project/tutorial/` に保存する

3. 付録Cの `index.html` を `src/main/webapp/htmx-lab/` に保存する
4. `pom.xml` があるフォルダで `mvn clean package` を実行し、`BUILD SUCCESS` を確認する
5. `target/multi-management-system/htmx-lab/index.html` と `target/multi-management-system/WEB-INF/classes/team/multi/project/tutorial/TaskServlet.class` があることを確認する
6. ServersパネルのTomcat 11から **Add Deployment** → **Exploded** で `target/multi-management-system` を選択する。登録済みなら二重登録しない
7. **Publish Server (Full)** を実行し、Tomcat 11を起動する
8. `http://localhost:8080/multi-management-system/htmx-lab/index.html` を開く
9. 挨拶と時刻を取得する
10. 初期タスクが2件表示されることを確認する
11. タスクを追加し、件数増加を確認する
12. 完了・未完了、編集・取消、削除を試す
13. 検索と状態フィルターを組み合わせる
14. DevToolsのNetworkでリクエストURL、HTMLレスポンス、`HX-Trigger` を確認する
15. `http://localhost:8080/multi-management-system/index.jsp` にアクセスし、既存アプリが従来どおり開くことを確認する
16. Tomcatを再起動し、教材用メモリデータが初期化されることを確認する

学習中の反映手順: 学習ブランチで `mvn package` → Serversパネルの **Publish Server**（必要ならFull）→ ブラウザー再読込。ソースコードを変更するだけではデプロイ済みのExplodedフォルダに自動反映されない場合がある。**学習終了後は付録Eに従って終了処理を行う。**

確認したい配置関係:

```
src/main/webapp/htmx-lab/index.html
          ↓ Mavenで展開
 target/multi-management-system/htmx-lab/index.html
          ↓ Explodedデプロイ
http://localhost:8080/multi-management-system/htmx-lab/index.html
```

注意事項

- 完成版は**HTMXの操作を学ぶための教材用アプリ**。ログイン、CSRF、DB永続化、ユーザーごとの認可は未実装であり、公開サーバーへそのまま配置しない
- 内部データは簡易的なスレッドセーフなコレクションで保持するが、検索と更新の間の厳密な一貫性、複数サーバー間の同期等は保証しない
- Servlet生成のHTMLではユーザー入力をエスケープしている。将来JSPへ変更するときも同じ原則を維持する
- `<form>` と `hx-*` を組み合わせた通常のフォールバックはPart 5の発展課題。完成版の `htmx-lab/index.html` はJavaScript有効を前提とする
- `api/records`、`api/upload`、`projects`、`tasks` 等の一部サンプルURLは**構文説明・発展課題用**であり、完成版サーバーには存在しない
- CDNが利用できない環境ではhtmx 2.0.11のJSをローカルに配置し、`src` をそのパスへ変更する

付録E — 学習終了後にブランチを削除し、`main` に戻す

目的: 学習用のGitコミットを `main` にマージせず破棄し、既存プロジェクトに戻る。実行するのは**HTMX学習を完全に終了し、学習用コードが不要になったときだけ**。

E.1 作業を整理する

まず今いるブランチと変更内容を確認する。

```
git branch --show-current
git status --short
```

表示ブランチが `study/htmx-hda` であることを確認する。作業ツリーに変更が残っている場合、**ブランチ削除だけでは未コミット変更や未追跡ファイルは消えない**。教材で追加・変更したファイルだけを選んで最後の学習用コミットにまとめる。

```
git add pom.xml src/main/java/team/multi/project/tutorial/ src/main/webapp/htmx-lab/
git diff --cached --stat
git commit -m "study: finish htmx tutorial"
git status --short
```

`web.xml` など上記以外のファイルを教材のために変更した場合は、**内容を確認してから**そのパスも `git add` する。コミット済みで差分がなければ再コミットは不要。`git status --short` が空になったことを確認する。他の作業が混ざっている場合は、ここで止めて変更を仕分ける。

> **なぜ削除前にコミットするのか**: コミットしていない変更はブランチ固有ではなく作業ツリーに存在する。対して、学習専用ブランチだけにコミットされた変更は、`main` に切り替えれば通常の作業ツリーから外れる。そのブランチ参照を削除することで学習履歴を通常のブランチ一覧から片付けられる。

E.2 `main` に戻してブランチを削除する

```
git switch main
git branch -D study/htmx-hda
git branch --show-current
git branch --list
git status --short
```

- `git switch main`: 学習ブランチではなく既存のチーム開発ブランチへ戻る
- `git branch -D ...`: **未マージの学習コミットがあっても**ローカルブランチを削除する。これは意図的な破棄のために限って使用する
- `git branch --list`: `study/htmx-hda` が一覧から消えたことを確認する

- `git status --short`: 未コミット・未追跡の学習用ファイルが残っていないか確認する

通常の `git branch -d study/htmx-hda` は未マージのブランチを削除しない。そのため、本教材では学習コミットを残さないことを確認したうえで `-D` を使う。**`git merge`、`git rebase`、Pull Requestによるマージは行わない**。

注意: `-D` はブランチ名を消す操作であり、機密情報の安全な消去を保証するものではない。不要な一時ファイルが残っていても、`git clean -fd` や `git reset --hard` を無条件に実行しない。他の作業ファイルまで消す危険がある。

E.3 Tomcatのデプロイ状態も戻す

Gitブランチを消しても、Mavenの `target/`、Tomcatの公開フォルダ、実行中のWebアプリが自動で以前の状態に戻るわけではない。**Gitで管理するソースと、ビルド・デプロイされたファイルは別**。

1. VS CodeのServersパネルからTomcat 11を停止する
2. 現在のブランチが `main` であることを確認する
3. `pom.xml` のあるフォルダで次を実行する

```
git branch --show-current
mvn clean package
```

4. Serversパネルで既存の `multi-management-system` のExplodedデプロイを **Publish Server (Full)** して、`main` からビルドした成果物を反映する
5. Tomcatを起動して `http://localhost:8080/multi-management-system/index.jsp` を確認する
6. `/htmx-lab/index.html` や `/htmx-lab/api/hello` に以前の学習画面が残っていないか確認する。残る場合は**教材用のデプロイ済みファイルだけ**を調べて整理する。既存のチームアプリ全体やTomcatの設定を誤って削除しない

`target/` が `.gitignore` に含まれていれば通常はGitのブランチ切り替えで消えないため、`mvn clean package` で再作成する。Tomcat側のPublish設定・キャッシュ等により古いファイルが残る場合は、再デプロイ状態を確認する。

E.4 リモートへプッシュした場合のみ

本教材ではローカルブランチだけを使い、**`git push` は不要**。学習ブランチを意図してGitHubへプッシュした場合だけ、リモート側のブランチ削除が別途必要となる。

```
git push origin --delete study/htmx-hda
```

チーム共用リポジトリでは、リモートブランチの所有者・用途を確認してから実行する。他の人が利用しているブランチは削除しない。

E.5 学習用ブランチのライフサイクル

```
[main: 作業ツリーがクリーン]
       ↓ git switch -c study/htmx-hda
[study/htmx-hda: 実装・ビルド・ローカルコミット]
       ↓ 学習終了／未コミット変更を整理
[git switch main]
       ↓ git branch -D study/htmx-hda
[main: 学習コミットは含まれない]
       ↓ mvn clean package → Tomcatを再Publish
[既存のチーム開発環境に復帰]
```

最終チェック: 現在のブランチは `main`、学習ブランチはローカル一覧に存在しない、`git status --short` は空、学習用Servlet・HTMLは `main` に存在しない、Tomcatは `main` のコードを実行している。

公式資料

- htmx Documentation（2.x）
- htmx Reference（2.x）
- Hypermedia-Driven Applications — htmx
- htmx Examples
- htmx 4.0 Documentation
- htmx 4.0 What's New
- Apache Tomcat 11 Documentation
- Jakarta Servlet 6.1 仕様
- Apache Tomcat バージョン対応表